Deployment Guidelines

Document ID: doc-003	Department: Engineering	**Sensitivity: INTERNAL**

Version 3.1 | Last updated: 2026-03-10 | Owner: Engineering Department

1. Purpose

This document defines the standard process for deploying application changes to staging and production environments across all engineering-owned services. It applies to every service in the company's service catalog, including internal tools, unless a service has an explicitly approved deviation documented in its own runbook.

2. Environments

Environment	Trigger	Purpose
dev	Every merge to main	Rapid iteration and integration testing
staging	Automatic after dev tests pass	QA validation and stakeholder sign-off
production	Manual, approved release pipeline	Serves live customer traffic

Each environment is provisioned identically (same infrastructure-as-code templates) with only scale and data differing, to minimize the risk of environment-specific deployment failures.

3. Pre-Deployment Checklist

- All CI checks (lint, unit tests, integration tests) must pass on the pull request.
- At least one peer approval is required before merge, from a code owner of the affected module.
- Database migrations must be backward-compatible and reviewed separately from application code changes.
- Feature flags should be used for any user-facing change that has not been fully validated in staging.
- A rollback plan must be documented directly in the deployment ticket before the release branch is cut.
- Any change touching authentication, authorization, or billing requires a security sign-off in addition to standard peer review.

4. Deployment Process

4.1 Standard Release Flow

- Create a release branch from main following the naming convention release/YYYY-MM-DD.
- Deploy to staging and run the full regression suite (automated + a short manual smoke test).
- Obtain sign-off from QA and the relevant product owner before proceeding.

- Trigger the production deployment pipeline; this requires a manual approval gate from the on-call engineer or team lead.
- Monitor key dashboards (error rate, latency, CPU/memory, queue depth) for at least 30 minutes post-deployment before considering the release complete.
- Update the deployment log with version, timestamp, deploying engineer, and any anomalies observed.

4.2 Hotfix Flow

For urgent production fixes, a hotfix branch may be cut directly from the current production tag. Hotfixes still require CI checks and at least one peer review, but the staging soak period may be reduced to 15 minutes with explicit on-call engineer sign-off, given the urgency.

5. Rollback Procedure

If error rates exceed 2% above baseline, latency exceeds agreed SLOs, or critical functionality breaks post-deployment, the on-call engineer must initiate an immediate rollback using the previous stable release tag.

- Rollbacks do not require the standard approval gate — speed takes priority during active degradation.
- The rollback must be reported in the #incidents channel within 15 minutes of initiation.
- A short retrospective note should be added to the deployment log explaining the rollback trigger.
- If the rollback itself fails or is insufficient, the incident escalates per the Incident Response Guide.

6. Change Freeze Windows

No non-critical deployments are permitted during the following windows, unless approved by the VP of Engineering:

- The last 3 business days of each fiscal quarter (quarter-end close).
- 24 hours before and during any major marketing launch or public company announcement.
- The week between December 24 and January 1 (year-end freeze), except for SEV-1 hotfixes.

7. Secrets and Configuration Management

All secrets (API keys, database credentials, signing keys) are managed via the internal secrets manager. Secrets must never be committed to version control, embedded in container images, or hardcoded in deployment scripts or infrastructure-as-code templates.

Environment-specific configuration is managed through environment variables injected at deploy time by the deployment pipeline, sourced from environment-scoped configuration stores. Any change to a production configuration value follows the same review process as a code change.

8. Infrastructure and Tooling

Layer	Tool / Approach
CI/CD pipeline	GitHub Actions with staged approval gates
Container orchestration	ECS Fargate (no self-managed Kubernetes for this scale of workload)
Infrastructure as Code	AWS CDK / CloudFormation
Monitoring & alerting	Centralized dashboards with threshold-based paging
Secrets management	Cloud-native secrets manager, least-privilege IAM access

9. Ownership and On-Call

Each service in the catalog must have a designated on-call owner responsible for deployment health and initial incident response for that service. On-call rotations are published weekly and on-call engineers are expected to acknowledge pages within 5 minutes for SEV-1/SEV-2 issues, consistent with the Incident Response Guide.

10. Deployment Metrics and Review

Engineering leadership reviews deployment health metrics monthly, including deployment frequency, change failure rate, mean time to recovery, and rollback rate. Services with a change failure rate above 15% over a rolling 3-month window are flagged for a dedicated reliability review.

11. Frequently Asked Questions

Can I deploy directly to production for a very small change?

No. All production changes, regardless of size, must go through staging first. The only exception is the hotfix flow described in Section 4.2, which still requires review and monitoring.

Who can approve the production deployment gate?

The on-call engineer for the affected service, or the team lead, may approve the gate. For services flagged as business-critical, a second approver from the reliability team is also required.

What if staging is unavailable?

Deployments should not proceed to production if staging validation could not be completed, except under an active SEV-1 incident, where mitigation takes priority and staging validation may be performed retroactively.

12. Service Tiering and Deployment Rigor

Not every service carries the same blast radius if a deployment goes wrong. Services are classified into tiers, and the rigor applied above scales with tier.

Tier	Definition	Extra Requirements
Tier 1	Customer-facing, revenue-critical (e.g., core app, billing).	Second approver on production gate; canary rollout requi
Tier 2	Internal-facing or supporting services with indirect customer impact.	Standard flow as described in Section 4.
Tier 3	Internal tooling with no customer impact.	Staging soak period may be waived with lead approval.

12.1 Canary Rollouts

Tier 1 services must deploy using a canary strategy: the new version is first routed a small percentage of production traffic (typically 5%), monitored against the same health thresholds described in Section 5, and only promoted to full traffic once those thresholds are met for at least 10 minutes.

13. Post-Deployment Review

For Tier 1 services, a brief post-deployment note is expected in the deployment log summarizing any anomalies observed during the monitoring window, even if the deployment was ultimately successful. This creates a lightweight historical record that has repeatedly proven useful during later incident investigations, without requiring a full postmortem for every release.

14. Policy Ownership

This document is owned by the Engineering Department and reviewed quarterly, or immediately following any incident where the deployment process itself was found to be a contributing factor.